K.R. Mangalam University

School of Engineering and Technology

Lab 3 - Generate visualizations and comparative charts

Smart Resource Allocation and Scheduling System using Greedy Algorithms

Program: BCA (AI & DS) Course Code - ENCA301 (2026-27)

Submitted by:

Name: Bhavya Rattan

RollNumber: 2401201004

Course: BCA (AI & DS)

Section: B

Submitted To:

Dr. Aarti Sangwan

Assistant Professor

A. Resource allocation: Fractional Knapsack

Items: Gold (10 kg, 60), Silver (20 kg, 100), Bronze (30 kg, 120), Platinum (15 kg, 105), Copper (25 kg, 50); capacity W = 50 kg. The greedy rule sorts by value-to-weight ratio (Platinum 7.0, Gold 6.0, Silver 5.0, Bronze 4.0, Copper 2.0).

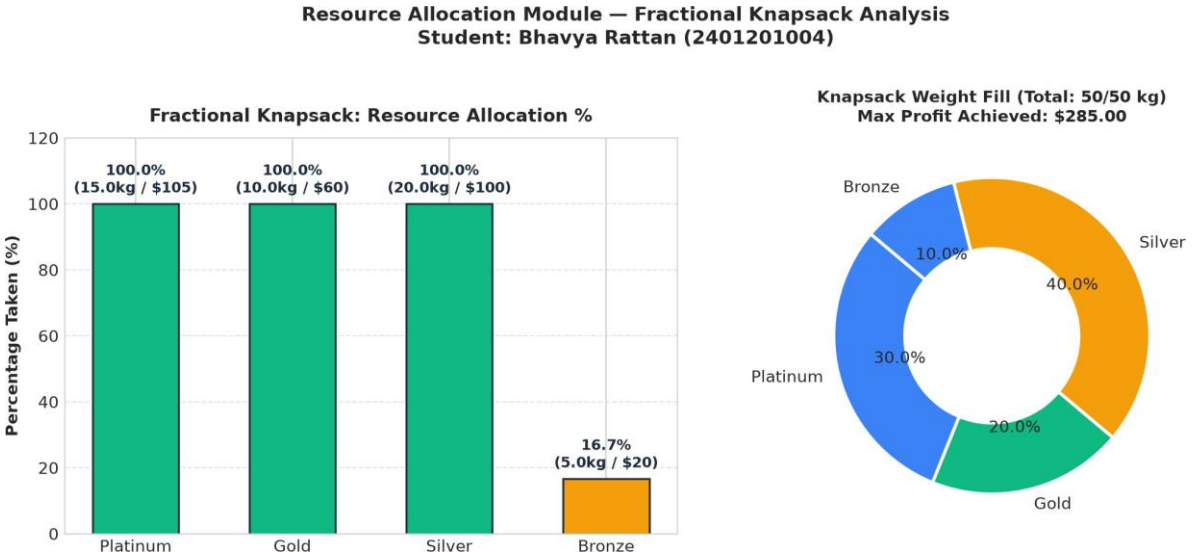


Figure 1: Fractional knapsack allocation: percentage of each item taken (left) and share of the 50 kg capacity (right).

Performance observations

- **Highest ratio first.** Platinum, Gold and Silver are taken whole (15 + 10 + 20 = 45 kg, value 105 + 60 + 100 = 265).
- **One fractional item.** The remaining 5 kg takes 16.7% of Bronze (5/30), adding value 20. Total maximum profit = **285.00**, with the knapsack exactly full (50/50 kg). Copper, the lowest ratio, is never taken.
- **Why it is optimal.** Because any fraction is allowed, swapping weight from a lower-ratio item to a higher-ratio one can never reduce value (exchange argument), so the greedy choice is globally optimal. The same rule is not optimal for the 0/1 knapsack.

B. Task scheduling: Activity Selection

Eight candidate activities (A1 [1,4], A2 [3,5], A3 [0,6], A4 [5,7], A5 [3,9], A6 [5,9], A7 [6,10], A8 [8,11]) are sorted by earliest finish time; an activity is selected only if its start is at or after the finish of the last selected one.

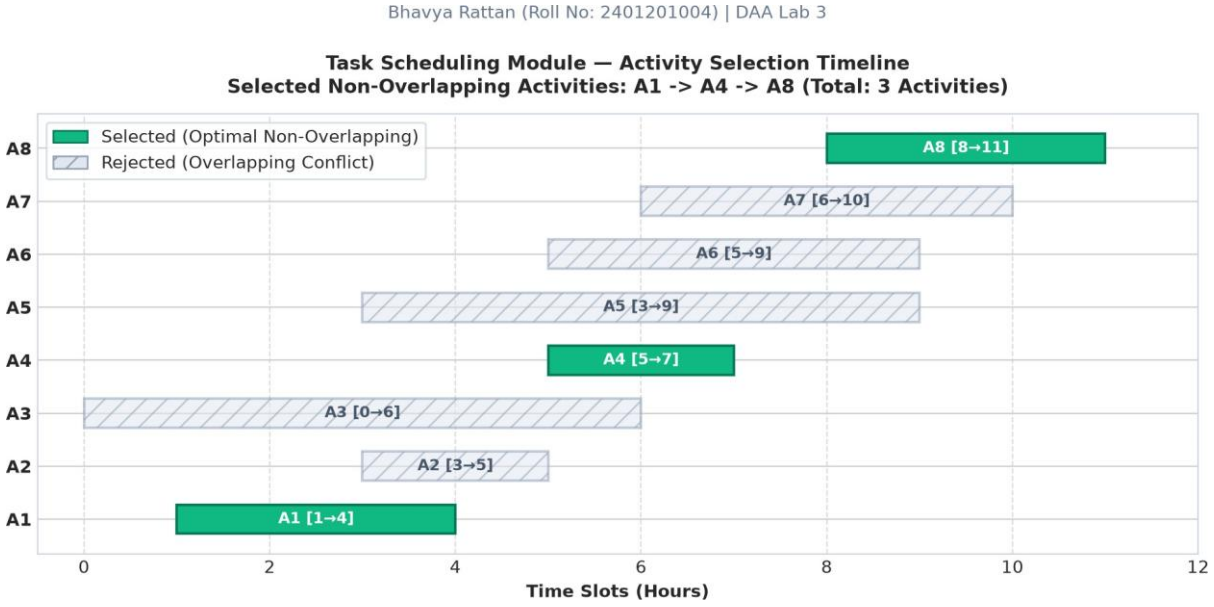


Figure 2: Activity selection timeline: selected (solid) and rejected (hatched) activities.

Performance observations

- **Result.** The greedy schedule is A1 [1-4], A4 [5-7], A8 [8-11]: **3 of 8** activities, with no overlaps.
- **Why the rest are rejected.** A2, A3 and A5 start before A1 finishes (time 4); A6 starts at 5, before A4 finishes (7); A7 starts at 6, also before A4 finishes. Finishing early leaves the most time free for later activities.
- **Cost.** Sorting dominates, so time is O(n log n); the single scan afterwards is O(n). Extra space is O(n) for the sorted copy and the result list.
- **Metric matters.** Shortest-duration-first fails (for [0,5], [4,6], [5,10] it picks only [4,6]); earliest-finish-first is provably optimal.

C. Data compression: Huffman Coding

Input string: **abracadabra** (11 characters, 5 distinct). The two lowest-frequency nodes are merged repeatedly with a min-heap.

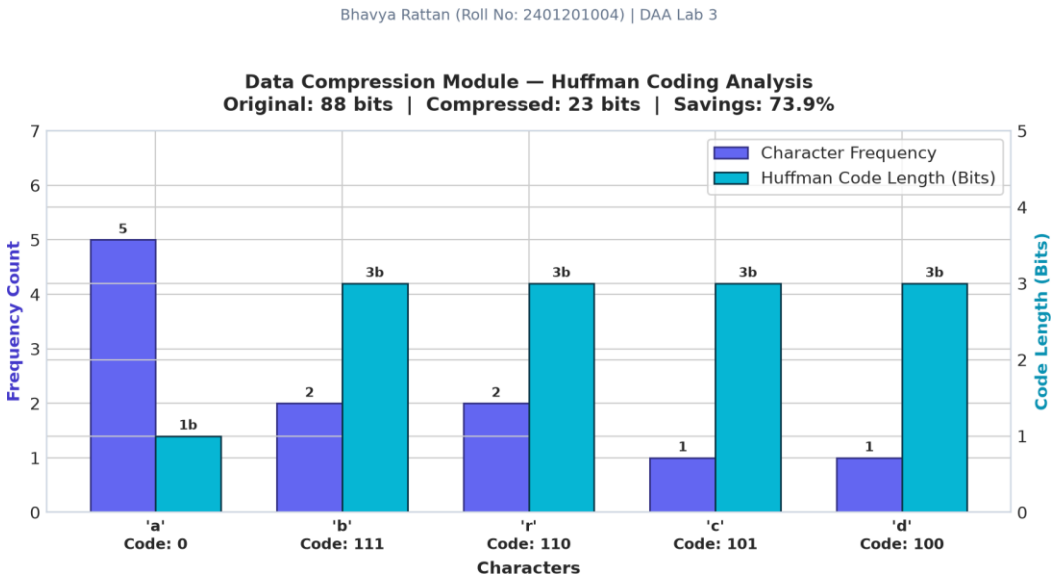


Figure 3: Character frequency versus assigned Huffman code length.

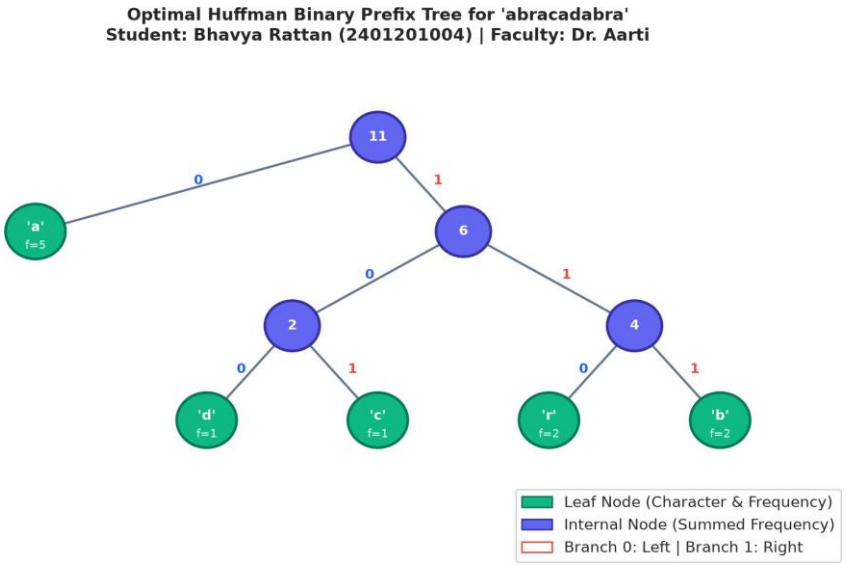


Figure 4: Huffman prefix tree for 'abracadabra' (left branch = 0, right branch = 1).

Performance observations

- **Short codes for frequent symbols.** 'a' (5 occurrences) gets the 1-bit code 0; 'b', 'r', 'c', 'd' get 3-bit codes (111, 110, 101, 100).
- **Compression.** Encoded length = 5x1 + 2x3 + 2x3 + 1x3 + 1x3 = **23 bits** against 88 bits in 8-bit ASCII (**73.9% saving**, average 2.09 bits per character). A fixed 3-bit code would need 33 bits, so the variable-length code is also 30% smaller than the best fixed-length code.
- **Prefix-free.** No codeword is a prefix of another, so the bit stream decodes unambiguously with a single left-to-right walk of the tree.

D. Empirical benchmarking

Each algorithm was run on random inputs of size n = 10, 50, 100, 250, 500, 1,000, 2,500, 5,000 and 10,000 (seed 42). Time was measured with **time.perf_counter()** and peak memory with **tracemalloc**. Raw data: **data_csv/benchmark_results.csv**.

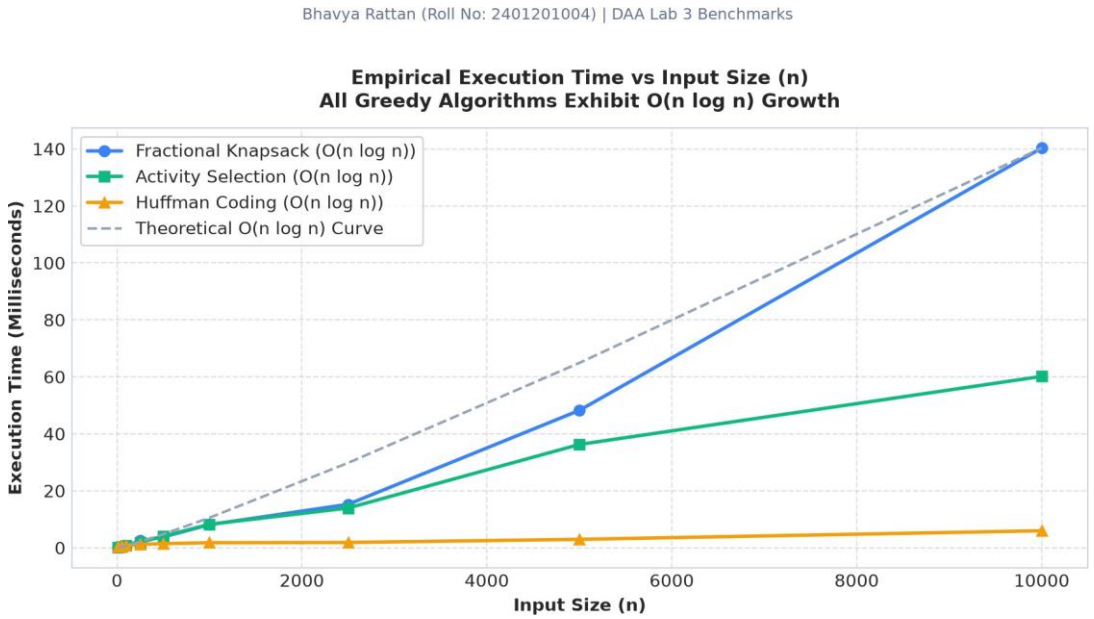


Figure 5: Execution time versus input size, with a theoretical O(n log n) reference scaled to the knapsack run at n = 10,000.

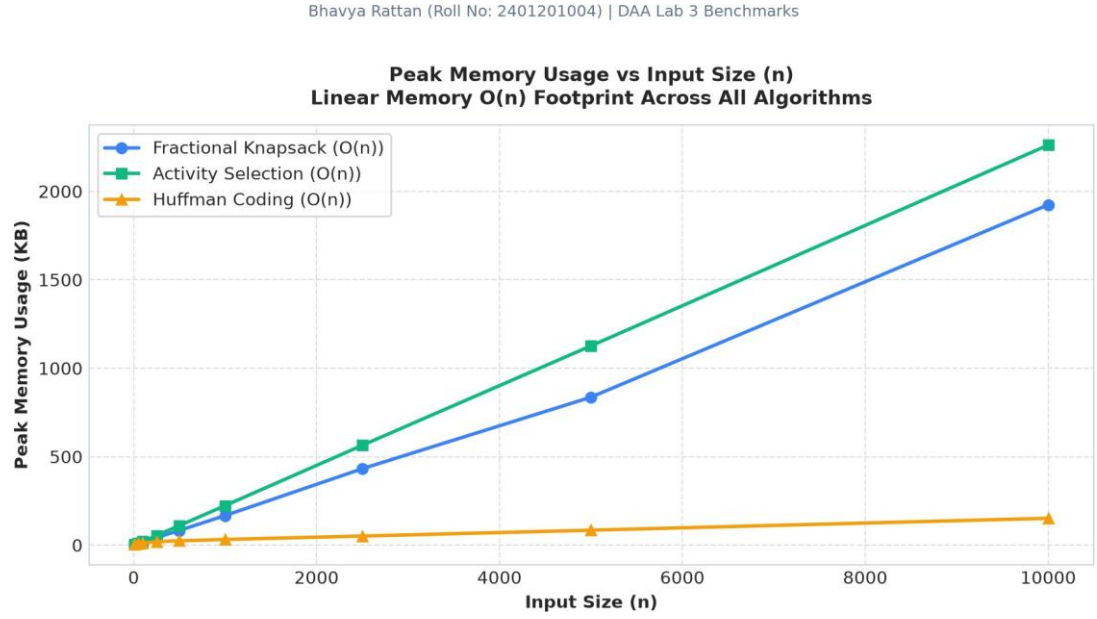


Figure 6: Peak memory versus input size.

Table 1: Measured execution time and peak memory.

n	Knapsack time (ms)	Knapsack mem (KB)	Activity time (ms)	Activity mem (KB)	Huffman time (ms)	Huffman mem (KB)
10	0.119	2.8	0.086	2.5	0.304	2.9
50	0.358	7.4	0.386	10.7	0.530	8.0
100	0.675	14.4	0.721	20.3	0.748	11.0
250	2.451	41.9	1.917	54.1	1.184	19.5
500	3.775	82.7	4.009	110.0	1.424	23.9
1,000	8.088	166.9	8.219	223.1	1.760	31.7
2,500	15.244	431.6	13.941	564.0	1.848	50.9
5,000	48.202	836.1	36.243	1,126.1	2.940	84.2
10,000	140.281	1,923.6	60.139	2,261.5	5.960	151.4

Performance observations

- **Knapsack and activity selection grow like n log n.** Fitted log-log time slopes are 1.01 (knapsack) and 0.96 (activity selection). From n = 100 to n = 10,000 (100x) knapsack time rose 208x and activity selection 83x; a pure n log n law predicts about 200x.
- **Memory is linear for both.** Fitted memory slopes are 0.98 and 1.00. At n = 10,000 knapsack peaked at 1,923.6 KB and activity selection at 2,261.5 KB, because each builds O(n) helper objects and a sorted copy.
- **Huffman grows much more slowly in n.** Time slope 0.39 and memory slope 0.54: at n = 10,000 it took 5.96 ms and 151.4 KB. The test strings use at most 62 distinct characters, so the heap and tree stay small (cost is about O(n + k log k) for k distinct symbols) and the main n-dependent work is a linear counting pass.
- **Measurement noise.** Timings below about 1 ms are noisy (for example Huffman is 1.85 ms at n = 2,500 against 1.76 ms at n = 1,000), so only the overall trend across sizes should be read from them.

E. Overall comparisons

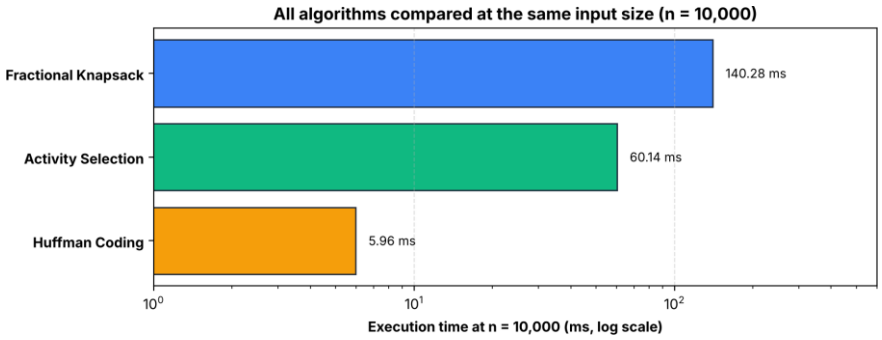


Figure 7: All three algorithms measured at the same input size (n = 10,000).

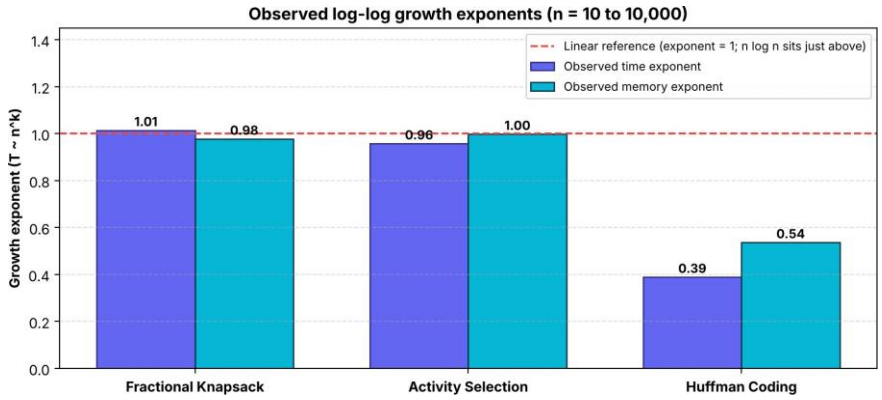


Figure 8: Observed log-log growth exponents for time and memory (n = 10 to 10,000).

Table 2: Complexity summary. For Huffman, n is the number of characters in the input; the heap work depends on the number of distinct symbols.

Algorithm	Greedy criterion	Time	Space	Result in this lab
Fractional Knapsack	Highest v/w ratio	O(n log n)	O(n)	Max profit 285.00 (W = 50)
Activity Selection	Earliest finish time	O(n log n)	O(n)	3 of 8 activities scheduled
Huffman Coding	Merge two lowest frequencies	O(n log n)	O(n)	88 to 23 bits (73.9% saved)

Overall observations

- At n = 10,000 the knapsack run (140.28 ms) is 2.3x slower than activity selection (60.14 ms) and 23.5x slower than Huffman (5.96 ms), even though all three are described as O(n log n). Constant factors, per-item object creation and, for Huffman, the small alphabet account for the gap.
- Each of the three greedy algorithms finishes 10,000 items in well under a quarter of a second and uses at most about 2.2 MB, so all are practical at this scale.
- Greedy is correct here because each problem has the greedy-choice property and optimal substructure. It does not carry over to 0/1 knapsack (W = 50, items 10/60, 20/100, 30/120: greedy gives 160, the optimum is 220), which needs dynamic programming.
- Real-world uses: fractional resource slicing in cloud computing (knapsack), room and CPU scheduling (activity selection), and entropy coding in JPEG, MP3 and ZIP (Huffman).